



FUNCTIONAL RAIL

SIMULATORE DI RETE FERROVIARIA IN SCALA

Amadori Nicolas, Nanni Denise

OBIETTIVI

MODELLAZIONE RETE

Stazioni (piccole/grandi), binari (normali/alta velocità).

DEFINIZIONE TRENI

Tipo, stazione di partenza, fermate.

DINAMICA DI SIMULAZIONE

Generazione viaggiatori, movimento treni, gestione precedenza e guasti.

RISULTATO

Report con statistiche su tempi di percorrenza ed efficienza della rete.



PROCESSO DI SVILUPPO

METODOLOGIA SCRUM

Sprint settimanali, planning e backlog.

VERSIONING

Git e GitHub per la gestione delle versioni secondo versioning semantico.

TESTING

Abbiamo implementato test automatici per verificare il corretto funzionamento del codice.

FORMATTING

Scalafix e Scalaformatter per l'uniformità dello stile del codice.

CI/CD

GitHub Actions per l'esecuzione di test automatici e creazione delle release.

TEST E TEST COVERAGE

I test sono stati sviluppati con ScalaTest, utilizzando lo stile AnyFlatSpec per renderli più espressivi.

Abbiamo verificato la copertura dei test tramite lo strumento Scoverage.

```
"Passengers" should "be generated and should be in a station" in {
  val PLAYER_NUMBER = 5
  val state = SimulationState(List(train1, train2))
  val passengerGenerator = PassengerGenerator(railway, state.trains)
  val (newState, newGenerator, logs) = state.generatePassengers(passengerGenerator)(PLAYER_NUMBER)

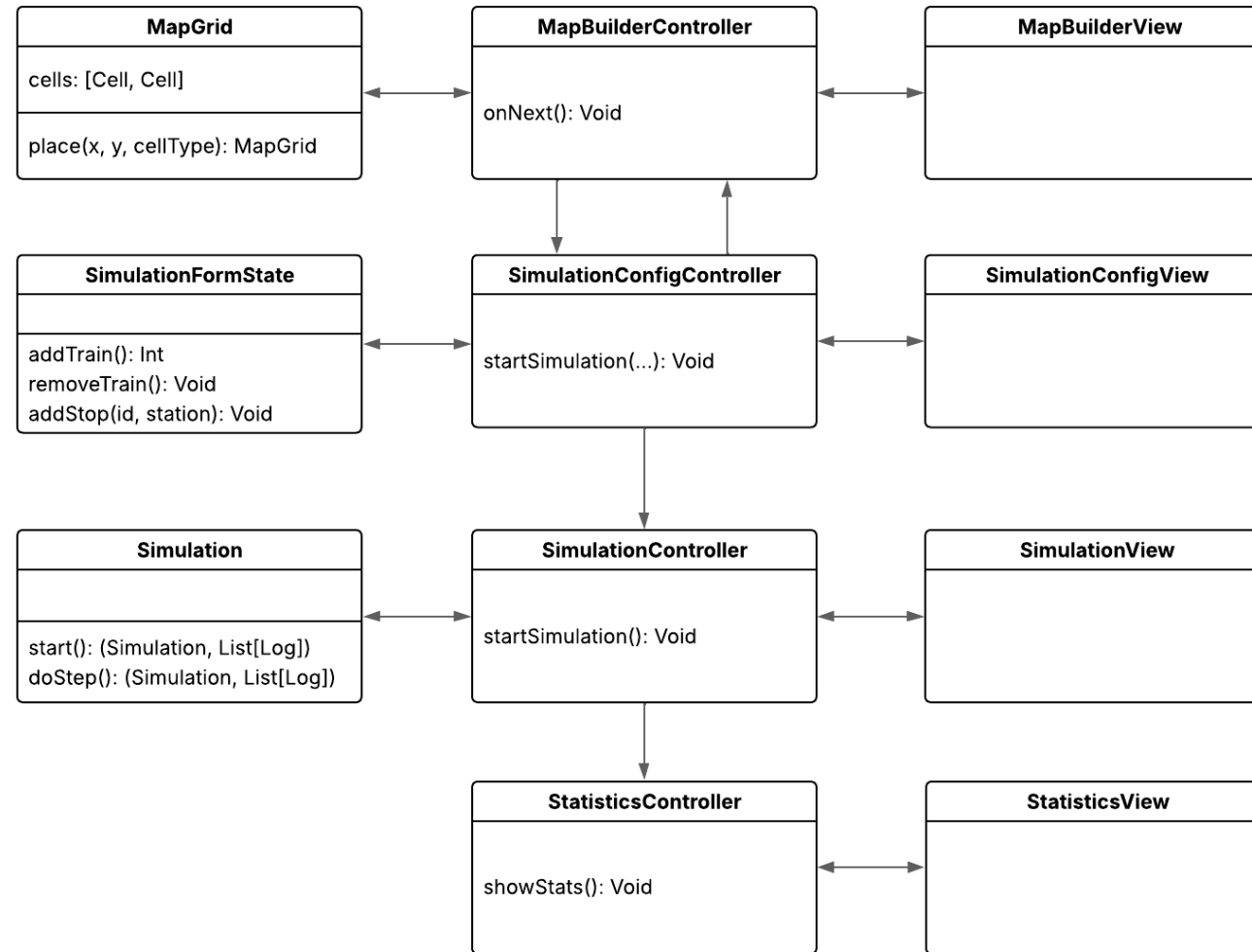
  newState.passengers.size should be(PLAYER_NUMBER)
  newState.passengerStates.size should be(PLAYER_NUMBER)
  logs.size should be(PLAYER_NUMBER)

  newState.passengerStates.foreach { (_, state) =>
    assert(state.currentPosition match
      | case PassengerPosition.AtStation(station) => stationCodes.contains(station)
      | case PassengerPosition.OnTrain(_) => false)
  }
}
```

```
it should "enter station when rail crossed" in {
  val train1 = buildNormalTrain(trainCode1):
    _ in railway departsFrom StationCode(StationA) stopsAt StationCode(StationB)
  val firstRail = train1.route.getRailAt(0)
  val travelTime = train1.getTravelTime(firstRail).ceil.toInt
  val simulation = createSimulationWithTrains(List(trainWithRoute1))
  val loopedSimulation = simulation.loopFor(setupLoop + travelTime + enterStationLoop)

  val state = loopedSimulation.state
  val trainState = state.trainStates(TrainCode(trainCode1))
  state.simulationStep should be(setupLoop + travelTime + enterStationLoop)
  state.railStates(firstRail.code).isFree should be(true)
  trainState.position.get should be(AtStation(firstRail.stationB))
}
```

DESIGN ARCHITETTURALE



MAPGRID

Rappresenta la rete ferroviaria in forma di griglia, controllando la coerenza dei posizionamenti.

RAILWAY

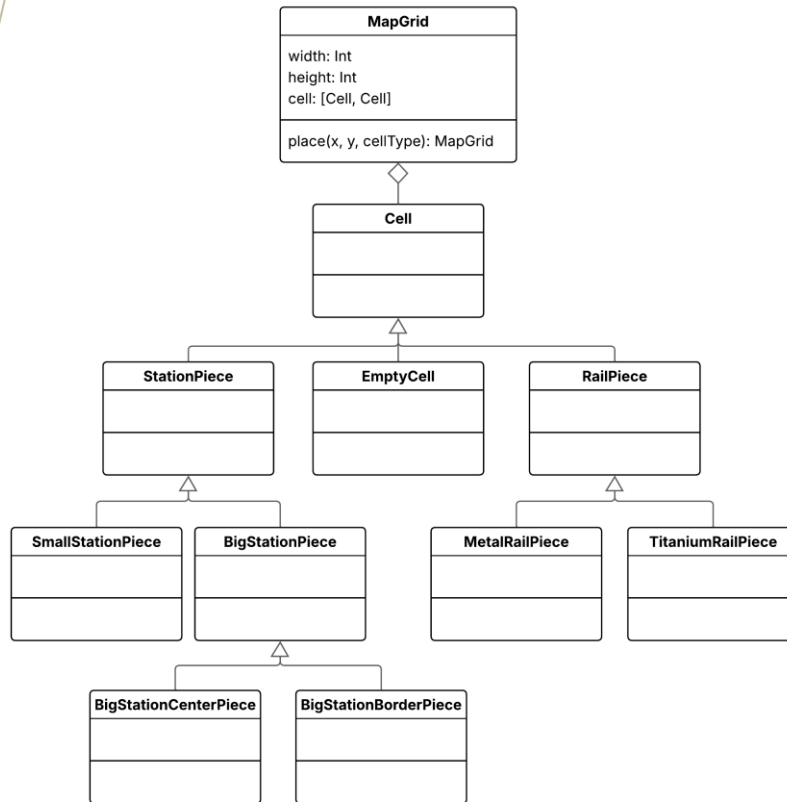
Modello statico della rete convertito da MapGrid, base per la configurazione e la simulazione.

SIMULATION

Gestisce lo stato dinamico della simulazione (treni, binari, passeggeri, eventi).

MODEL

MAPGRID



RAPPRESENTAZIONE A GRIGLIA

Stazioni, binari e celle vuote con regole di posizionamento e vincoli di budget.

IMMUTABILITÀ

Ogni modifica crea una nuova istanza, preservando lo stato precedente.

VALIDAZIONE INTEGRATA

Controllo correttezza della rete senza gestire logica di treni o passeggeri.

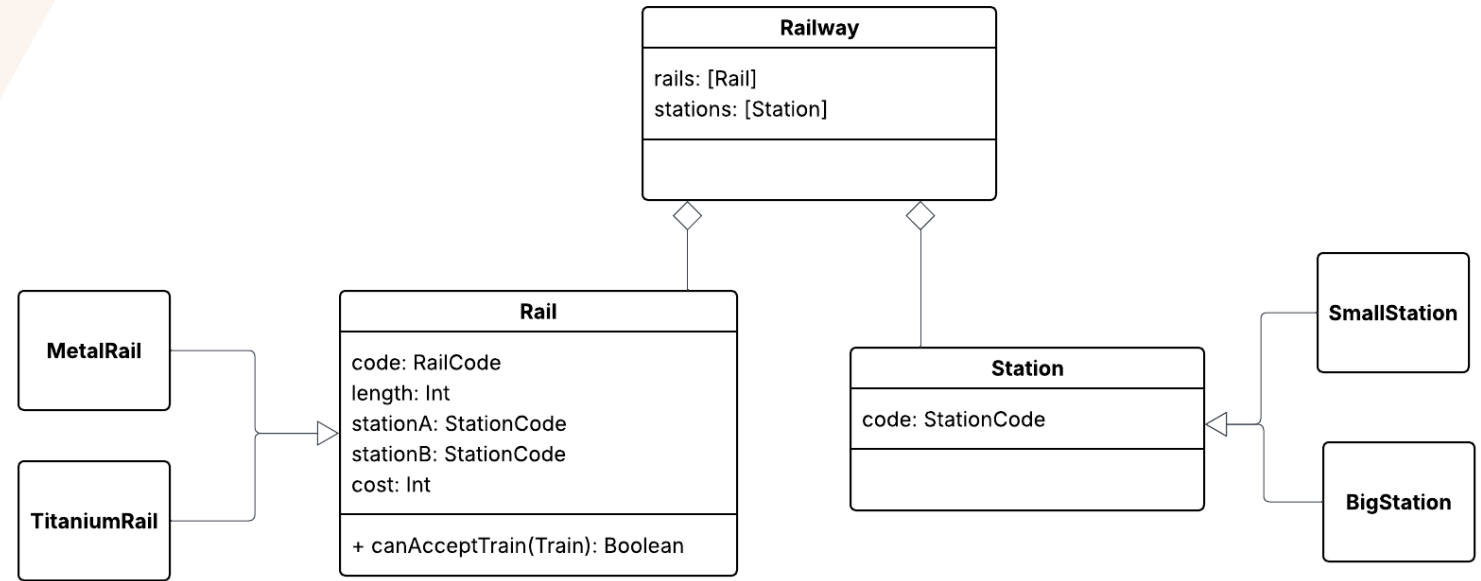
ESTENDIBILITÀ

Nuovi tipi di stazioni/binari ereditano da *Cell* senza modificare la struttura base.

OUTPUT

La griglia viene convertita in un modello *Railway* utilizzato dalla simulazione.

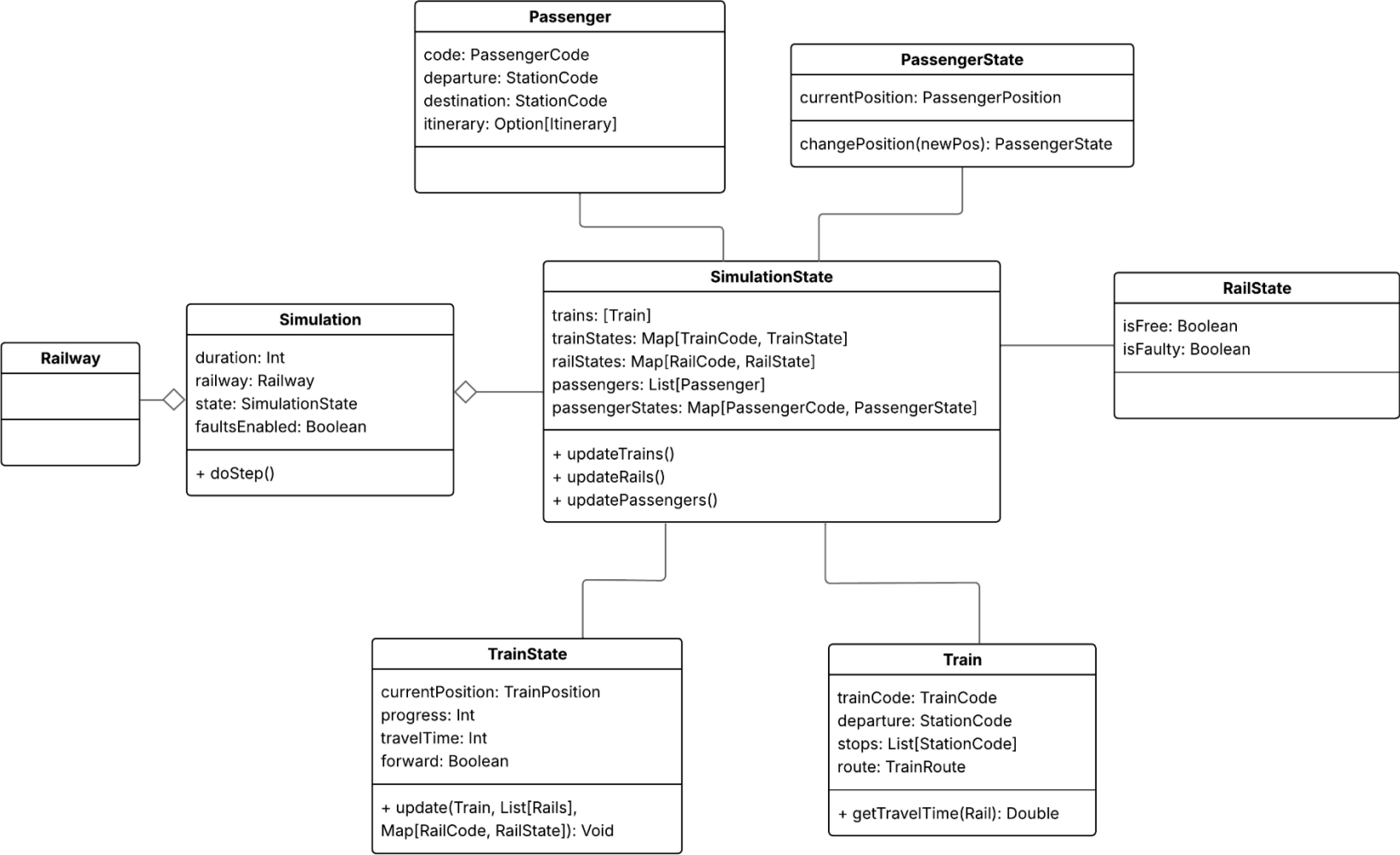
RAILWAY



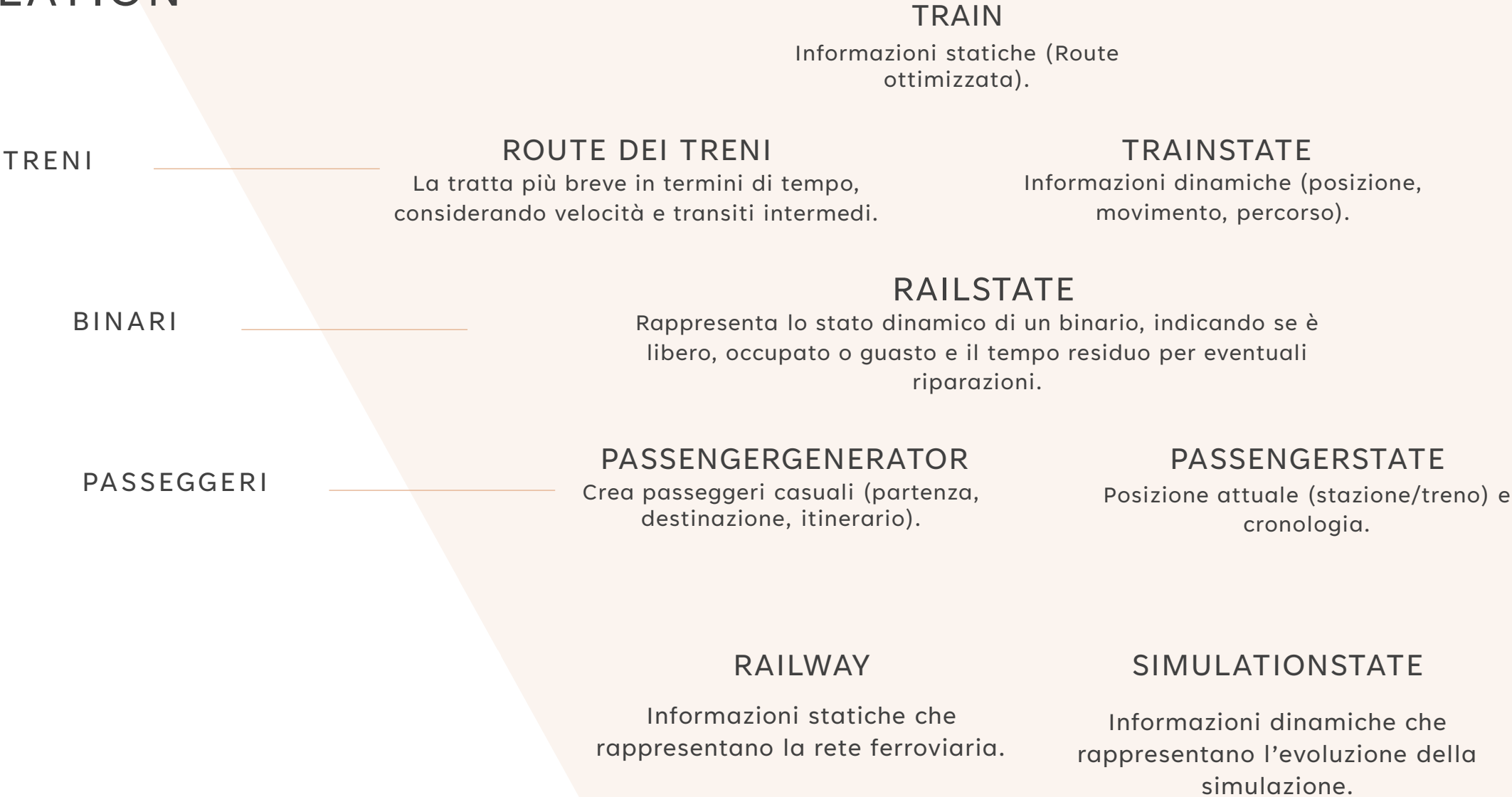
STRUTTURA A GRAFO

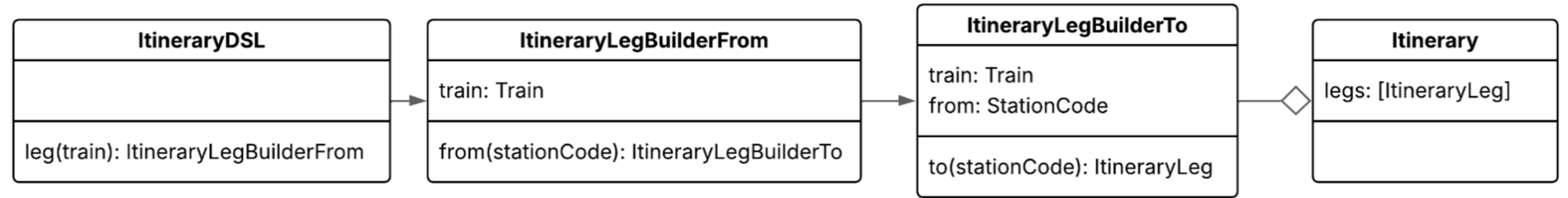
La modellazione statica della rete è facilmente convertibile in un grafo per il calcolo delle tratte.

SIMULATION



SIMULATION





PASSENGERDSL

- Creazione controllata di Passenger ispirato al Builder pattern
- Possibilità di associare un itinerario o lasciare il passeggero senza percorso se non disponibile

TRAINDSL

- Creazione di Train con Builder pattern
- Validazione automatica e assegnazione delle tratte

ITINERARYDSL

- Creazione di ItineraryLeg ispirato al Builder pattern
- Generazione di leg effettuando la loro validazione

DSL

